

VIZAC

Design, Development and Evaluation of VIZAC: A WordPress-Based Client Portal for Consultancy Application Management

Updated Technical Project Report based on MSc Computer Science project work

Project version documented: VIZAC Client Portal v1.0.0

Muhammad Salman

The University of Law

MSc Computer Science

September 2026

Project focus

A practice-based software engineering project examining how WordPress can be extended into a client-facing consultancy portal with user-specific application tracking, branching workflow logic and an auditable status history.

About this report

This document is an updated technical report documenting VIZAC, a software project originally developed during MSc Computer Science study at The University of Law and subsequently restored, reviewed and extended. It is based on the implemented WordPress system, the custom VIZAC Client Portal plugin, development screenshots and documented testing evidence. It is not presented as an identical copy of the report originally submitted for university assessment.

The report is intended for technical review, portfolio use and research discussion. Implemented functionality is distinguished from proposed future work, and claims are limited to evidence that can be supported by the current system and development record.

Public repository: <https://github.com/itxmsalman/vizac-client-portal/>

Contents

Abstract	4
1. Introduction	4
2. Literature Review	5
3. Project Aim, Research Questions and Objectives	9
4. Methodology	10
5. Work Plan and Development Process	12
6. System Design and Implementation	14
7. Testing and Evaluation	21
8. Results and Discussion	23
9. Conclusion	24
10. Recommendations and Future Work	25
References	25
Appendix A. Public Repository and Reproducibility Notes	26

Abstract

Digital consultancy services increasingly depend on online platforms to communicate with clients, collect application information and coordinate service activity. In visa and study-abroad consultancy, however, clients may still have limited visibility of case progress and can become dependent on repeated messages or telephone calls for updates. This project addresses that operational problem through the design, development and evaluation of VIZAC, a WordPress-based consultancy client portal that combines a public service website with authenticated client functionality and application-progress management. The evaluated build was operated in a local WordPress development environment and was iteratively restored, tested and improved.

The platform provides public consultancy information, registration and login, a client profile, an application-intake form, service and appointment access, and a dedicated application-tracking page. A purpose-built VIZAC Client Portal plugin implements the core tracking logic. It reads an Advanced Custom Fields (ACF) application status associated with the logged-in WordPress user, maps the value to a client-friendly progress model, supports both successful and refusal branches, and displays explanatory messages appropriate to each outcome. The plugin also records status transitions in user metadata, including the previous status, new status, timestamp and the WordPress user responsible for the update, producing a visible case-activity timeline.

The project followed an Agile-inspired iterative Software Development Life Cycle (SDLC), informed by design-science principles. Evaluation used developer-led functional testing, compatibility checks and structured review against usability, maintainability, privacy and web-application security considerations. The finished prototype demonstrates that a general-purpose content management system can be extended into a lightweight consultancy case-management environment when standard WordPress functionality is combined with controlled user metadata, configurable plugins and bespoke application logic. The project also identifies limitations, including the absence of formal penetration testing, large-scale user evaluation, automated notifications and a complete document-management workflow. These limitations provide a clear route for future development and for further research into secure, transparent and human-centred digital case-management systems.

1. Introduction

Digital service delivery has changed expectations about how organisations communicate with customers. Users increasingly expect to complete tasks online, obtain timely information and understand the current state of a service without depending on a staff member for every routine update. These expectations are particularly relevant in consultancy settings where a case may move through several stages over days or weeks and where uncertainty can cause repeated contact between clients and consultants. A well-designed client portal can therefore perform two roles at once: it can provide a self-service information channel for the client and reduce avoidable administrative communication for the consultancy.

Visa and study-abroad consultancy provides a useful example of this problem. A consultancy may hold up-to-date case knowledge, but clients do not necessarily have direct access to that information in a structured form. Telephone calls, instant messages and email can communicate individual updates, yet they create fragmented records and place the burden on staff to answer the same question repeatedly: what is happening with my application? A digital portal cannot replace professional advice or an official government case-management system, but it can provide a controlled view of the consultancy's current understanding of a client's case and indicate the next expected stage.

Research on digital service design supports the importance of self-service capability, convenience and transparency. Chan, Brown and Venkatesh (2021) describe transparency as a design characteristic that helps users understand how a digital service works and follow a request through its life cycle. Their analysis of e-government services also links self-service, accessibility, security, support and transparency with perceived service quality and satisfaction. These ideas are transferable to a consultancy portal: a client does not need access to internal systems, but should be able to understand what stage has been recorded, what that stage means and whether action is required.

VIZAC was developed to explore this problem in a practical WordPress environment. The project began as a consultancy website and evolved into a more specialised client portal. The current system includes public-facing pages such as Home, About, Services, Portfolio and Contact; authenticated registration and login; a user profile; an application-intake form; service and appointment functionality; and a user-specific application tracking interface. The strongest custom contribution is the VIZAC Client Portal plugin, which separates application-tracking logic from the active theme and provides status presentation and audit-history functionality through WordPress hooks, user metadata and shortcodes.

The move from theme-level customisation to a standalone plugin is significant from a software-engineering perspective. WordPress documentation recommends implementing site functionality through plugins rather than modifying core files, and plugin-based separation also reduces the risk that an unrelated theme update will remove business logic. The resulting architecture is therefore not only a visual website but a small information system with multiple actors, persistent state, access boundaries and workflow history.

The project is also relevant to information security and data governance. Application forms and client profiles may contain personal and potentially sensitive information. The UK Information Commissioner's Office (ICO) states that personal data should be processed with appropriate technical and organisational measures and that access should be appropriately authenticated, authorised and auditable. Similarly, the OWASP Top 10 identifies broken access control and identification/authentication failures as major web-application risks. VIZAC does not claim to be a production-certified secure system; instead, security is treated as a design requirement, an evaluation dimension and a source of explicit limitations for future development.

This report presents an updated technical account of the design, development and evaluation of VIZAC, a project originating from MSc Computer Science study. It focuses on the problem addressed, the academic and technical foundations informing the solution, the architecture and implementation of the portal, the custom tracking plugin, the testing performed, and the limitations that would need to be addressed before production deployment. The report is intentionally evidence-based: it distinguishes implemented functionality from proposed future work and does not treat the prototype as an official source of government visa-status information.

2. Literature Review

2.1 Digital self-service, transparency and client confidence

Digital self-service is valuable because it enables users to obtain information or complete routine tasks without waiting for direct staff intervention. In service systems, however, self-service is only useful when users can understand the process and trust the information presented. Chan et al. (2021) identify accuracy, completeness, self-service capability and convenience as characteristics of the core digital service, while privacy protection, security protection and user support act as facilitating characteristics. They also treat transparency as a supporting characteristic that allows users to follow a service request

across its life cycle. This provides a useful conceptual basis for VIZAC: the portal should not simply display a generic 'processing' message; it should expose an intelligible sequence of stages and preserve enough history for the client to understand how the case reached its current state.

Related e-government research has repeatedly emphasised information quality, system quality and service quality. Papadomichelaki and Mentzas (2012) developed the e-GovQual scale to assess the quality of online public services, while later research has continued to identify functionality, ease of use, assistance, privacy/security and information quality as important dimensions of electronic service quality. Although a consultancy website is not an e-government service, the user problem is similar in one important respect: an individual is interacting with a digital interface in order to understand an administrative process. This justifies adapting ideas such as process visibility, self-service and information clarity to the VIZAC context.

The design implication is that progress information should be meaningful rather than decorative. A percentage alone can be misleading if it is not tied to identifiable stages. VIZAC therefore combines a progress bar with named states: Application Submitted, Under Review, Approved and Visa Issued. A refusal is modelled as an alternative terminal branch rather than as a normal stage between approval and issuance. This reflects the underlying business meaning more accurately and demonstrates the broader principle that interface elements should represent domain logic rather than merely create visual movement.

2.2 Technology acceptance, system success and e-service quality

The Technology Acceptance Model (TAM) provides a long-established explanation of why users accept or reject information systems. Davis (1989) identifies perceived usefulness and perceived ease of use as fundamental determinants of user acceptance. For VIZAC, perceived usefulness is closely connected to whether the portal gives clients information that reduces uncertainty or unnecessary contact, while perceived ease of use depends on whether clients can register, sign in, navigate to their account and interpret status information without specialist technical knowledge.

The DeLone and McLean Information Systems Success Model adds a broader evaluation perspective. DeLone and McLean (2003) distinguish system quality, information quality, service quality, use, user satisfaction and net benefits. These dimensions are valuable for an applied project because they discourage evaluating success only by asking whether the code runs. A tracking page could be technically functional but still provide poor-quality information, confusing messages or little practical benefit. The VIZAC evaluation therefore considers functional correctness together with clarity, client relevance, maintainability and potential operational benefit.

Electronic service quality research makes a similar point. Parasuraman, Zeithaml and Malhotra (2005) developed E-S-QUAL around efficiency, fulfilment, system availability and privacy, with responsiveness and contact becoming particularly important when users encounter problems. These categories influenced the project in two ways. First, the public and authenticated areas were designed to provide clear navigation and visible contact options. Second, the tracking interface includes status-specific explanatory text so that clients are not left to infer the meaning of a stage from a coloured icon alone.

2.3 Usability and interface consistency

Usability concerns whether users can achieve intended goals effectively, efficiently and with acceptable effort. Brooke's System Usability Scale (SUS) is widely used as a lightweight instrument for obtaining a global view of perceived usability, although a formal SUS study was not conducted for this project. The underlying principle remains relevant: interfaces should avoid unnecessary complexity, inconsistent behaviour and excessive learning requirements. VIZAC adopts a consistent dark visual design with a lime-

green accent across public pages and the authenticated experience, reducing abrupt visual changes when users move between marketing content, account functions and the tracking page.

The screenshots of the completed system show a repeated navigation model, clear page titles and action-oriented buttons. The login and registration pages use focused forms rather than embedding authentication inside a dense content page. The application form groups personal and application-related fields into a two-column layout on desktop. The tracking page places current status, percentage and stage indicators at the top, followed by a separate case-activity area. This hierarchy reflects the priority of the user's questions: What is my current status? What does it mean? What happened previously?

Usability is not only a visual matter. The underlying status labels must be stable and predictable. VIZAC uses a small controlled set of status values that are mapped to user-facing labels. This reduces the chance that minor wording differences create inconsistent progress behaviour. The plugin also derives its visible label from the ACF field choices when available, allowing the administrative configuration and client display to remain aligned.

2.4 Security, privacy and accountability

A consultancy portal processes user accounts, profile data and application information, so confidentiality and access control are central non-functional requirements. The ICO's current UK data-protection guidance states that organisations should assess information-security risk, appropriately authenticate and authorise users, constrain unnecessary access and maintain appropriate audit trails. The principle is particularly relevant where a portal stores passport or application information because unauthorised access could cause material harm.

OWASP's Top 10 places broken access control at the top of its 2021 web-application risk list and also identifies identification and authentication failures as a major category. These risks informed the project's decision to make the application-progress shortcode dependent on WordPress authentication and to derive the displayed user record from `get_current_user_id()` rather than from a user identifier provided in the page URL. This does not constitute a full security proof, but it reduces one obvious path by which a client might attempt to request another user's tracking record.

The custom plugin also uses common WordPress/PHP defensive practices. Values written to the audit history are passed through sanitisation functions, numeric user IDs are normalised with `absint()`, and dynamic output is escaped before rendering. In addition, the status-history record stores who made each change and when. This is valuable not only for client transparency but also for accountability: if the same case is updated by multiple staff accounts, the record can support later review.

Important security work remains outside the scope of the prototype. No formal penetration test, threat model, multi-factor authentication evaluation or independent code review was carried out. The project also does not claim application-level encryption of user metadata or a complete data-retention policy. These are treated as explicit limitations rather than silently assumed controls.

2.5 WordPress as an extensible application platform

WordPress is commonly described as a content management system, but its plugin architecture, user model, metadata APIs and hook system allow it to support more specialised application behaviour. The WordPress Plugin Developer Handbook explains that plugins extend functionality without modifying WordPress core and that hooks allow code to respond to or modify events in WordPress. The Shortcode API provides a mechanism for rendering custom dynamic content inside normal pages, while the user metadata model provides a practical persistence layer for small amounts of user-specific state.

Advanced Custom Fields (ACF) complements this model by allowing administrators to define structured fields and retrieve field objects and values through PHP. In VIZAC, the `application_status` field acts as the administrative source of the client's current case status. The custom plugin retrieves the field configuration, reads the current user's stored value, maps the value to business logic and renders the result. An ACF update filter is also used to capture transitions before the new value replaces the previous value.

The choice of WordPress has trade-offs. It accelerates development by providing authentication, content management, themes and a large plugin ecosystem, but it also introduces dependency management and compatibility risk. The BookingPress issue encountered after restoring the project illustrates this: service records existed, yet the frontend booking form failed until the plugin was updated. This is a reminder that a plugin-based architecture must be maintained as a system, not as a collection of independent components.

2.6 Research gap and design implications

Existing literature provides strong guidance on digital service quality, self-service, transparency, usability and security, but it does not prescribe a single implementation pattern for a small consultancy using WordPress. The practical gap addressed by this project is therefore an integration problem: how can a low-cost, widely available CMS be adapted to support client-specific case visibility without building a complete custom case-management platform from the ground up?

From the literature and the operational problem, five design implications were derived. First, the system should provide user-specific rather than generic progress information. Second, progress should be modelled using meaningful domain states, including alternative outcomes. Third, the interface should explain each state in plain language. Fourth, status changes should be auditable. Fifth, business logic should be separated from theme presentation so that the functionality remains maintainable. These implications became the core requirements for the VIZAC Client Portal plugin.

2.7 Socio-technical perspective and workflow governance

A further perspective is that a consultancy portal is not simply a technical interface. It changes how information is produced, interpreted and communicated between staff and clients. The accuracy of the portal depends on a consultant selecting an appropriate state, while the usefulness of the state depends on the client understanding what that state means. This makes the system socio-technical: technical controls, organisational practices and human interpretation interact to determine whether the service is trustworthy.

The status model therefore acts as a small form of workflow governance. It constrains the available client-facing states and gives each state a defined presentation. This can reduce ambiguity compared with unstructured messages, but it can also create false confidence if staff update statuses inconsistently or if a label is interpreted as an official authority decision when it is only the consultancy's recorded understanding. Governance requires both technical representation and organisational rules about who may update a case, when an update should be made, what evidence supports the change, and how exceptional situations are handled.

The audit history supports this governance objective by preserving transitions rather than overwriting all evidence of the previous state. Even a lightweight history can improve traceability because a later reviewer can see that a case moved from one status to another and identify the account responsible for the change. Nevertheless, the current history is not an immutable security log: it is stored as WordPress user metadata and administrators with sufficient privileges could alter database content. A production-grade audit design would therefore require stronger integrity controls, event logging and retention rules.

This distinction between an informative history and a tamper-resistant audit log is important in evaluating the system accurately.

This socio-technical view also strengthens the relevance of VIZAC to later research. The project demonstrates a recurring information-systems problem: organisations need digital processes that are usable enough to reduce friction, transparent enough to build confidence, and governed strongly enough to preserve accountability. These competing requirements are central to many cybersecurity and digital-governance problems even when the application itself is not primarily a security product.

3. Project Aim, Research Questions and Objectives

The overall aim of the project was to design, implement and evaluate a WordPress-based consultancy portal that improves client visibility of application progress while integrating with a broader service website. The project was deliberately scoped as a lightweight consultancy information system rather than an official government-tracking service.

The primary research and development question was: How can WordPress be extended to provide a maintainable, user-specific consultancy portal that communicates application progress and preserves a clear history of status changes? Two supporting questions were used during development: (1) how should the application workflow be represented so that successful and refused outcomes remain understandable to clients; and (2) what privacy, access-control and maintainability considerations arise when implementing client-specific workflow logic inside a plugin-based CMS?

3.1 Project objectives

- Develop a coherent consultancy platform that combines public service information with authenticated client functionality.
- Provide client registration, login and profile functionality so that application information can be associated with a stable user identity.
- Provide a structured application-intake interface for personal and application-related information.
- Provide access to consultancy services and appointment booking within the same user journey.
- Implement a user-specific application-progress view driven by an administratively maintained status field.
- Represent successful and refusal outcomes using domain-appropriate branching rather than a single linear sequence.
- Record status changes with previous value, new value, timestamp and updater identity, and present the history to the client.
- Separate custom business logic from the active theme by packaging the tracking functionality in a dedicated WordPress plugin.
- Evaluate the prototype through functional testing and structured review of usability, privacy, security and maintainability.

3.2 Functional requirements

ID	Requirement	Description
FR1	Account access	Users can register and sign in to access authenticated functionality.
FR2	Client profile	Authenticated users have a persistent profile/account context.

ID	Requirement	Description
FR3	Application intake	Users can submit structured personal and application information.
FR4	Service access	Users can view consultancy services and access appointment booking.
FR5	User-specific tracking	The tracking page displays status for the currently logged-in user.
FR6	Workflow representation	Submitted, review, approved, issued and refused states are represented clearly.
FR7	Branching	Refusal is represented as an alternative outcome rather than a normal success stage.
FR8	Status guidance	Each major state provides explanatory client-facing text.
FR9	History	Status changes are appended to a visible timeline with date/time and updater.
FR10	Maintainability	Tracking logic is packaged as a plugin rather than tied to the active theme.

3.3 Non-functional requirements

ID	Quality attribute	Requirement
NFR1	Usability	Consistent navigation, readable status labels and clear calls to action.
NFR2	Privacy	Client-specific data should not be exposed to unauthenticated users or other clients.
NFR3	Security	Inputs stored by custom code should be sanitised and dynamic output escaped.
NFR4	Auditability	Material status changes should retain previous value, new value, time and updater.
NFR5	Maintainability	Business logic should survive theme changes and be separable from presentation.
NFR6	Compatibility	The system should be retested after plugin/core restoration or update.
NFR7	Extensibility	The custom component should provide a path toward additional workflow features.

4. Methodology

The methodology combined a structured software-development process with an applied research perspective. The system was built and refined as a working artefact, while evaluation focused on whether

the implemented workflow behaved as intended and what limitations remained. This approach was selected because the project concerned the design and improvement of an operational information system rather than the testing of a single causal hypothesis.

4.1 Software Development Life Cycle (SDLC)

A Software Development Life Cycle (SDLC) provides a structured way to organise software work across activities such as planning, design, implementation, testing, deployment and maintenance. AWS describes SDLC as a framework that divides software development into manageable stages and notes that iterative and Agile models allow requirements and software versions to evolve across repeated cycles (Amazon Web Services, 2026). For VIZAC, a purely sequential Waterfall process was not appropriate because the project involved restoring an existing WordPress installation, diagnosing compatibility problems and refining functionality as evidence emerged from the running system.

4.2 Agile-inspired iterative development

The development process therefore used an Agile-inspired iterative SDLC. The Agile Manifesto emphasises working software and responsiveness to change (Beck et al., 2001). These principles were applied pragmatically through repeated cycles of inspection, implementation, testing and refinement. The project was not managed as a formal Scrum implementation, and it does not claim Scrum ceremonies, formal sprint metrics or a product backlog that were not documented. Instead, the term Agile-inspired is used to describe the incremental way in which the system evolved in response to verified technical needs.

The main iterations were: restoration and access verification; checking public pages and authenticated account functions; diagnosing the BookingPress service-display problem; reconstructing the missing application-progress shortcode; confirming application_status as the authoritative ACF field; refining the success and refusal branches; moving business logic from the active theme into a dedicated plugin; adding automatic status-history recording; and repeating functional tests after each significant change. This sequence allowed defects and architectural weaknesses to be addressed when they became visible rather than postponing all testing until the end of development.

4.3 Design-science framing

The project was also informed by design-science research. Hevner et al. (2004) describe design science in information systems as the building and evaluation of artefacts intended to address relevant organisational problems. VIZAC fits this framing because the principal artefact is a working client portal and custom WordPress plugin, while the practical problem is limited client visibility and repeated communication in consultancy case management. The design-science perspective does not convert the project into a controlled experiment; rather, it provides a research-oriented rationale for connecting problem definition, artefact construction and evaluation.

4.4 Requirements and scope definition

Requirements were derived from the operational use case, the existing VIZAC platform and direct inspection of the restored system. A formal questionnaire was not conducted and no questionnaire results are claimed. The core client requirements were to register and sign in, submit relevant application information, access services, view only their own application progress, understand the meaning of the current stage and review a history of status changes. From the consultancy perspective, staff needed to update a client status through the WordPress administration environment without editing code for each individual case.

The scope was deliberately limited to a lightweight consultancy portal. The project did not attempt to reproduce a government case-management platform, provide real-time authority integration, support multiple concurrent applications per client, or provide production-grade document management. These boundaries helped keep the implementation aligned with demonstrable requirements and reduced the risk of presenting future ideas as completed functionality.

4.5 Evaluation strategy

Evaluation combined developer-led functional testing, integration checks and structured qualitative review. Functional tests examined authentication behaviour, current-user status resolution, submitted/review/approved/issued states, the refusal branch, status-history creation, timeline rendering, shortcode registration and booking availability. Integration checks were especially important because the system depends on WordPress core, the active theme, ACF, BookingPress and custom plugin code. The restored booking issue demonstrated that stored data can remain intact while frontend functionality is affected by plugin compatibility.

Usability was reviewed against task clarity, navigation consistency and information hierarchy rather than through a formal participant score. Security and privacy were reviewed against recognised concerns in ICO guidance and the OWASP Top 10, including authentication, access control, output handling, auditability and the handling of personal data. Because no independent participant study, penetration test or formal accessibility audit was conducted, the resulting evaluation is appropriately described as developer-led evidence rather than independent validation.

4.6 Ethical, privacy and responsible-development considerations

The portal can collect personal and application-related information, including fields that may be sensitive in context. Development and evaluation therefore used a local environment and public project materials should exclude real client records, credentials, database exports, complete WordPress backups and unredacted personal documents. A production deployment would require a documented lawful basis for processing, privacy information, data-minimisation and retention rules, appropriate role-based access, secure file handling, backup and recovery procedures, and proportionate security testing.

4.7 Evidence, validity and reproducibility

The evidence strategy is intentionally conservative. Claims in this report are limited to behaviours that can be supported by the implemented system, source code, screenshots and documented functional tests. Benefits suggested by the literature, such as improved transparency or reduced support demand, are treated as expectations for future empirical study rather than measured outcomes of the current prototype. This avoids overstating what the available evidence can demonstrate.

The custom plugin, status values, WordPress hooks and functional test cases also provide a degree of technical reproducibility: another developer can inspect the implementation and repeat the principal workflow tests. Reproducibility is necessarily partial because the complete local WordPress environment and third-party plugin configuration are not published as a public database or site backup for privacy, licensing and security reasons.

5. Work Plan and Development Process

Consistent with the Agile-inspired SDLC described in Chapter 4, the project was organised into iterative phases rather than a fixed one-pass sequence. This better reflected the reality of extending an existing WordPress site where compatibility issues, restored configuration and custom code all had to be verified.

The work plan below represents the logical development sequence used for the current build; several activities were revisited as testing exposed new requirements or maintenance issues.

Phase	Activity	Output
1	Problem definition and restoration	Restore the WordPress project, confirm access and identify broken functions.
2	Platform verification	Check public pages, login/registration, profiles, forms, services and booking.
3	Tracking reconstruction	Identify the missing shortcode logic and confirm the authoritative ACF status field.
4	Workflow refinement	Implement submitted, review, approved, issued and refusal states with client guidance.
5	Plugin separation	Move custom tracking logic from theme functions to VIZAC Client Portal plugin.
6	Audit history	Record status transitions and render a client-visible timeline.
7	Functional testing	Validate status paths, login gating, history, shortcode output and booking.
8	Documentation and evaluation	Document architecture, limitations, security considerations and future work.

6. System Design and Implementation

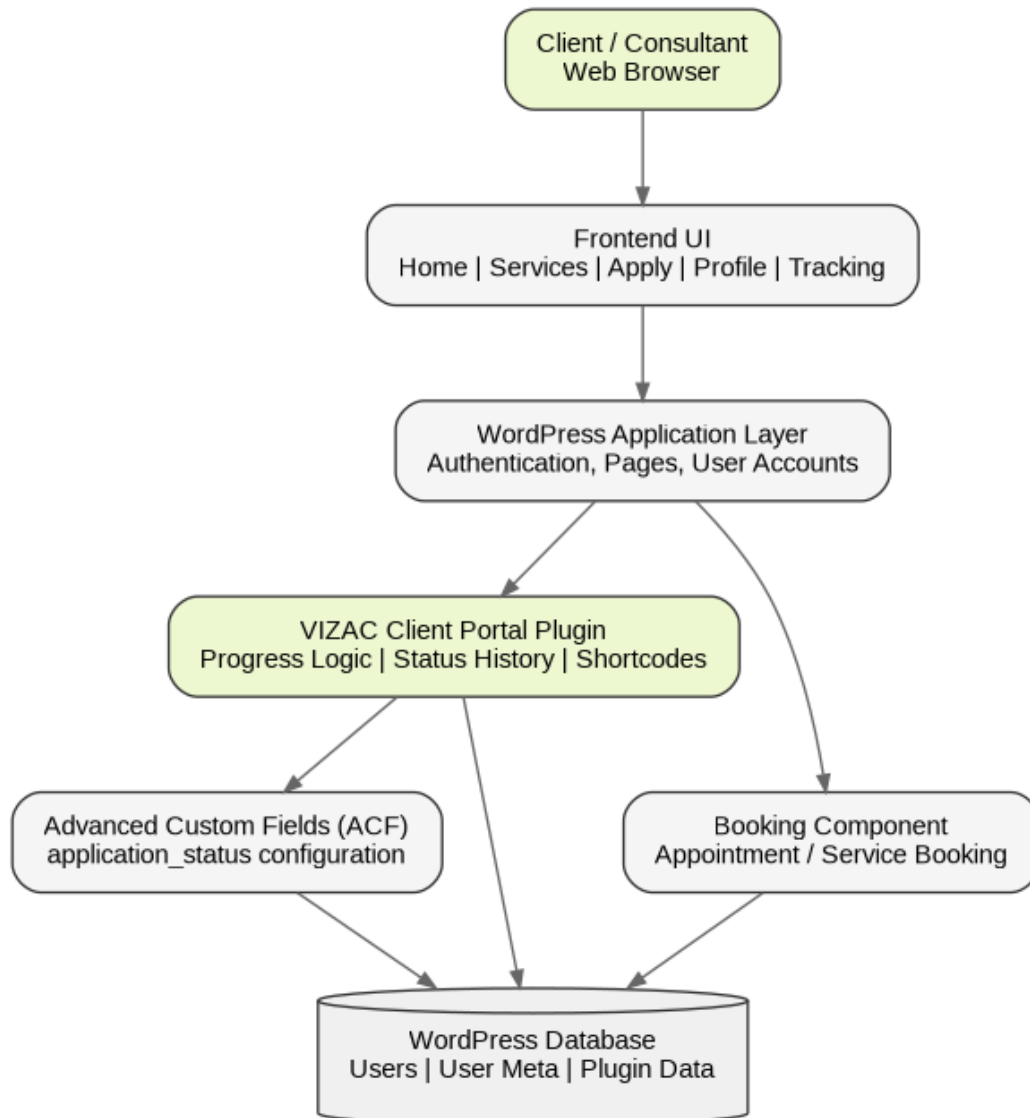


Figure 1. High-level architecture of the VIZAC platform.

6.1 System architecture

The VIZAC architecture combines standard WordPress capabilities, configurable plugins and bespoke PHP logic. The browser-facing interface contains public pages and authenticated pages. WordPress provides the base application layer, including routing, users, sessions and content. The custom VIZAC Client Portal plugin adds the domain-specific tracking behaviour. ACF stores the configured application-status field, while WordPress user metadata stores both the current application status and the custom history array. Appointment functionality is provided by the booking component. This architecture allowed the project to concentrate custom code on the parts that were unique to the consultancy workflow instead of rebuilding generic CMS functionality.

A major design decision was to use the logged-in WordPress user as the context for application tracking. The shortcode checks `is_user_logged_in()`, obtains `get_current_user_id()`, and then reads the

application_status value for that user. This avoids requiring clients to manually enter an application identifier on the tracking page and reduces the risk of accidental cross-user data display caused by a user-controlled URL parameter.

6.2 Public consultancy experience

The public website presents a coherent consultancy identity before the user enters the authenticated area. The homepage includes service categories, consultancy information, calls to action and testimonials. Separate About, Services, Portfolio and Contact pages provide additional information, while the navigation exposes Login and Register options. The completed design uses a dark background, white typography and a bright lime-green accent that is repeated across action buttons, headings and status indicators.

The Services page supports the operational purpose of the site by grouping offerings such as study-abroad advice, visitor services and work-permit queries and by providing a route to book a meeting. This public layer matters to the portal because it establishes context and provides a transition from initial enquiry to an authenticated client relationship.

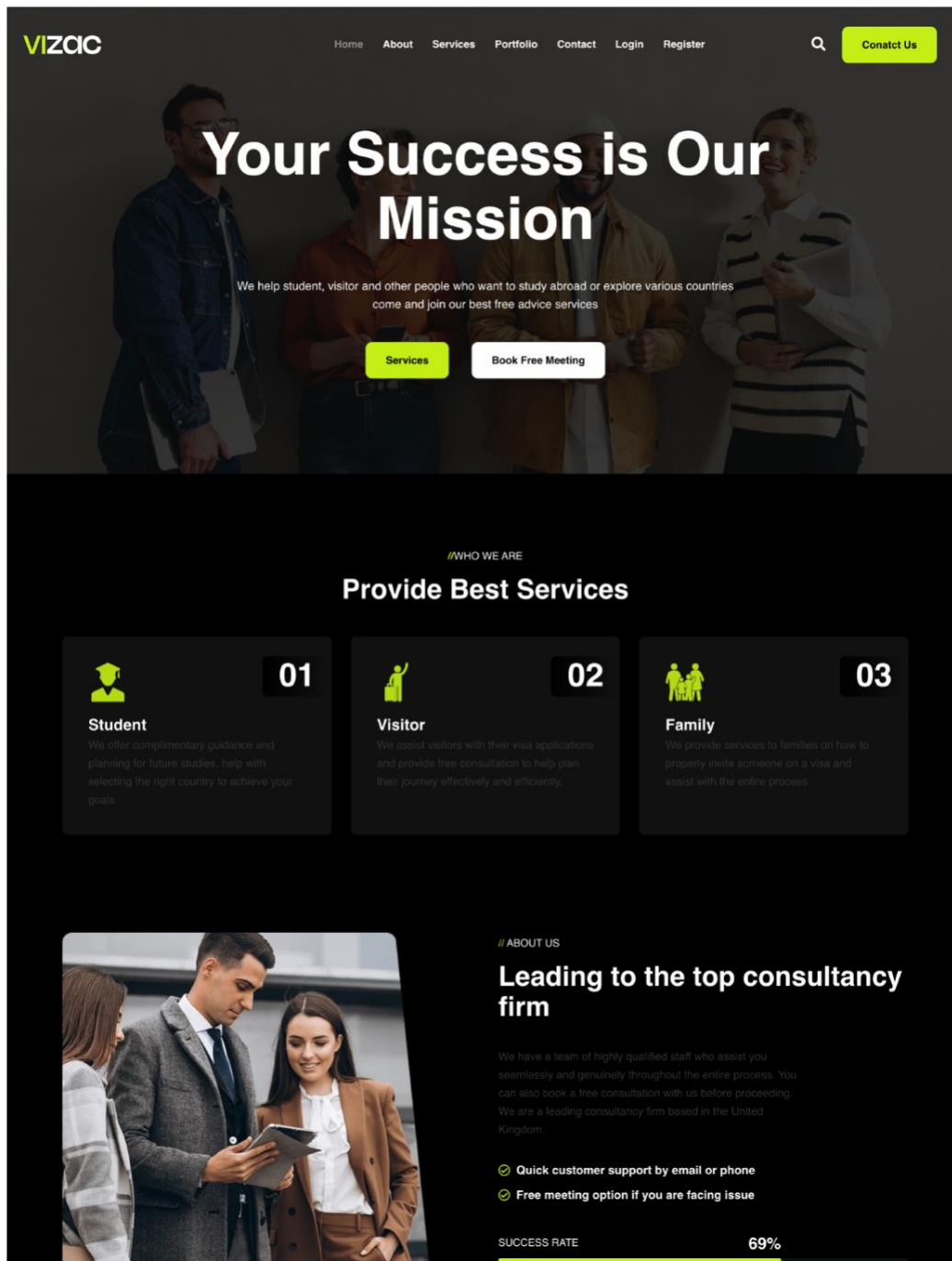


Figure 2. VIZAC public homepage showing consultancy positioning, services and calls to action.

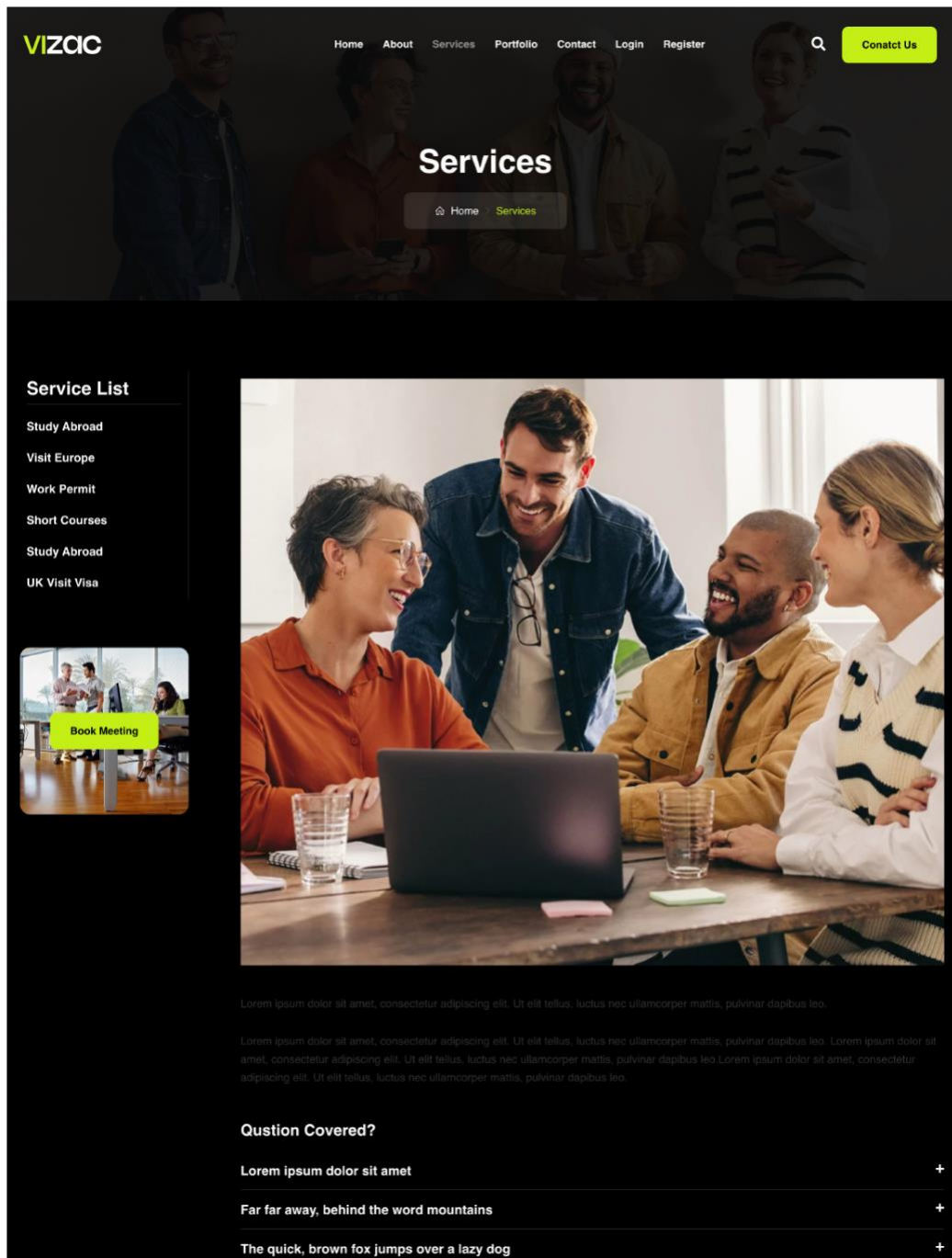


Figure 3. Services page showing consultancy categories and booking entry point.

6.3 Registration, login and client profile

Dedicated registration and sign-in pages provide the entry point to the authenticated experience. Registration collects account credentials and includes a privacy-policy confirmation. The login page presents username/email and password fields in a focused interface. Once authenticated, navigation changes to show account and logout options rather than login and register links.

The client profile provides a persistent account area containing basic user information and links. From a system perspective, the importance of the profile is not the visual profile card itself but the existence of a

stable WordPress user identity. The tracking system uses that identity to resolve the relevant application status and history without asking the client to select a case.

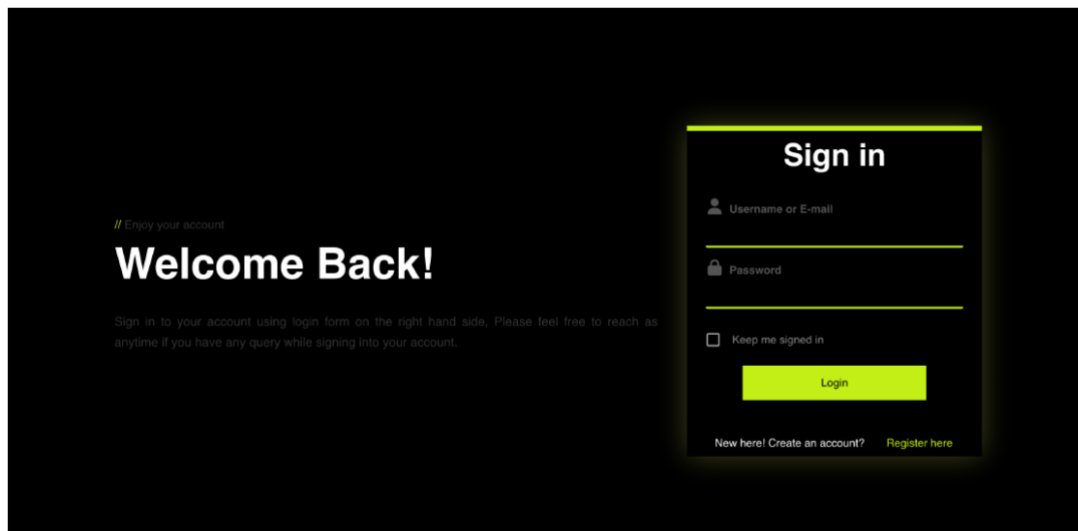


Figure 4. Dedicated client sign-in interface.

6.4 Application intake

The Apply Now page collects structured application-related information. The visible form includes fields for full name, date of birth, gender, nationality, place of birth, country of residence, passport number, passport expiry date, email, mobile number, visa type, employment status, country preference, education level, travel history and additional information, together with a file input and a CAPTCHA-style challenge. This creates a single entry point for information that would otherwise be fragmented across messages or attachments.

The report does not assume that the current form is suitable for unrestricted production use. Several of the fields may contain high-value personal data, so production deployment would require a documented retention period, secure file handling, administrator access rules and appropriate privacy notices. The current implementation is best understood as a functional prototype of the intake workflow.

6.5 Appointment and service booking

Appointment booking extends the portal beyond static information by allowing users to request consultancy interactions. During restoration testing, the booking page initially displayed no available services even though the BookingPress administration interface contained configured services. The database and category configuration were inspected and found to be present. Updating the plugin restored the frontend form. This issue is important because it demonstrates a real maintenance characteristic of CMS systems: correctness depends on compatibility between WordPress core, themes and active plugins as well as on stored data.

The incident informed the final maintenance recommendation that plugin versions and compatibility should be treated as part of system configuration and tested after site restoration or migration.

6.6 Custom application-progress plugin

The core contribution of the current project is the VIZAC Client Portal plugin, documented as version 1.0.0 in the public project repository. Earlier tracking logic had been placed in the active theme's functions.php file. This created a maintenance risk because replacing or updating the theme could remove business

functionality. The logic was therefore moved into a dedicated plugin with its own plugin header and shortcode registration. This follows the WordPress principle of separating site functionality from presentation and provides a cleaner foundation for future reuse.

The plugin registers the shortcodes [application_progress_bar] and [display_application_progress_bar]. When rendered, the function first checks that the user is logged in and that ACF field functions are available. It then loads the application_status field object for the current user and reads the stored user metadata. The field choices are used to produce the visible status label where possible, while tolerant string checks identify whether the state represents submission, review, approval, issuance or refusal.

Each state maps to a set of stages, a current-stage index, a percentage and a main colour. A submitted application is represented as 25% complete, review as 50%, approval as 75% and visa issued as 100%. Refusal is treated differently: the successful route is replaced with Submitted -> Under Review -> Visa Refused, and the progress indicator becomes red. The percentage is shown as a decision reached rather than implying that refusal is simply the final step of a successful journey.

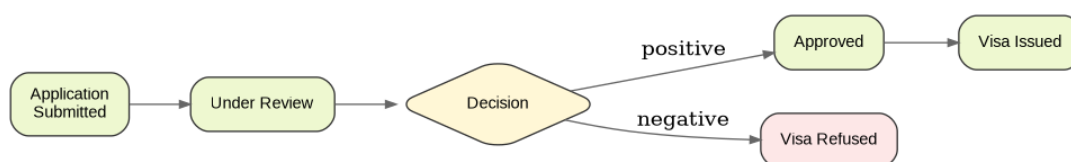


Figure 5. Application workflow represented by the custom tracking logic.

6.7 Status-specific communication

The visual progress bar is accompanied by explanatory text. Under Review explains that the application is being considered and that the consultant will update the page when there is a development. Approval displays a congratulatory message and states that visa issuance is expected within approximately 24 hours, subject to the relevant authority completing final issuance. Visa Issued indicates that the application process is complete. Refusal displays a clear red warning and asks the client to provide the complete refusal decision letter so that the consultancy can review the reasons and advise on next steps.

These messages are intentionally written as consultancy guidance rather than as claims of direct government integration. The system communicates the status maintained by the consultancy team. It does not claim to poll, scrape or connect to an official immigration authority in real time. This distinction is important for both accuracy and professional responsibility.

6.8 Application status history and audit trail

The second custom feature is automatic status-history recording. The plugin attaches a callback to the ACF update_value filter for the application_status field. Before ACF saves the new value, the callback reads the existing user metadata. If the new and old values differ, it retrieves the current history array, appends a record and writes the array back to user metadata. Each entry stores previous_status, new_status, changed_at and changed_by.

The frontend renderer reverses the stored array so that the newest activity appears first, maps internal values to friendly labels, formats the date and resolves the updater's WordPress display name. Refusal entries are highlighted red while normal transitions use the VIZAC lime-green accent. The resulting timeline gives the client more context than a single current-status label and provides the beginnings of an audit trail for the consultancy.

The history should be interpreted correctly. It records transitions from the moment the feature became active; it does not reconstruct the exact dates on which older statuses were first set before history tracking existed. This limitation is important when using the audit data as evidence.

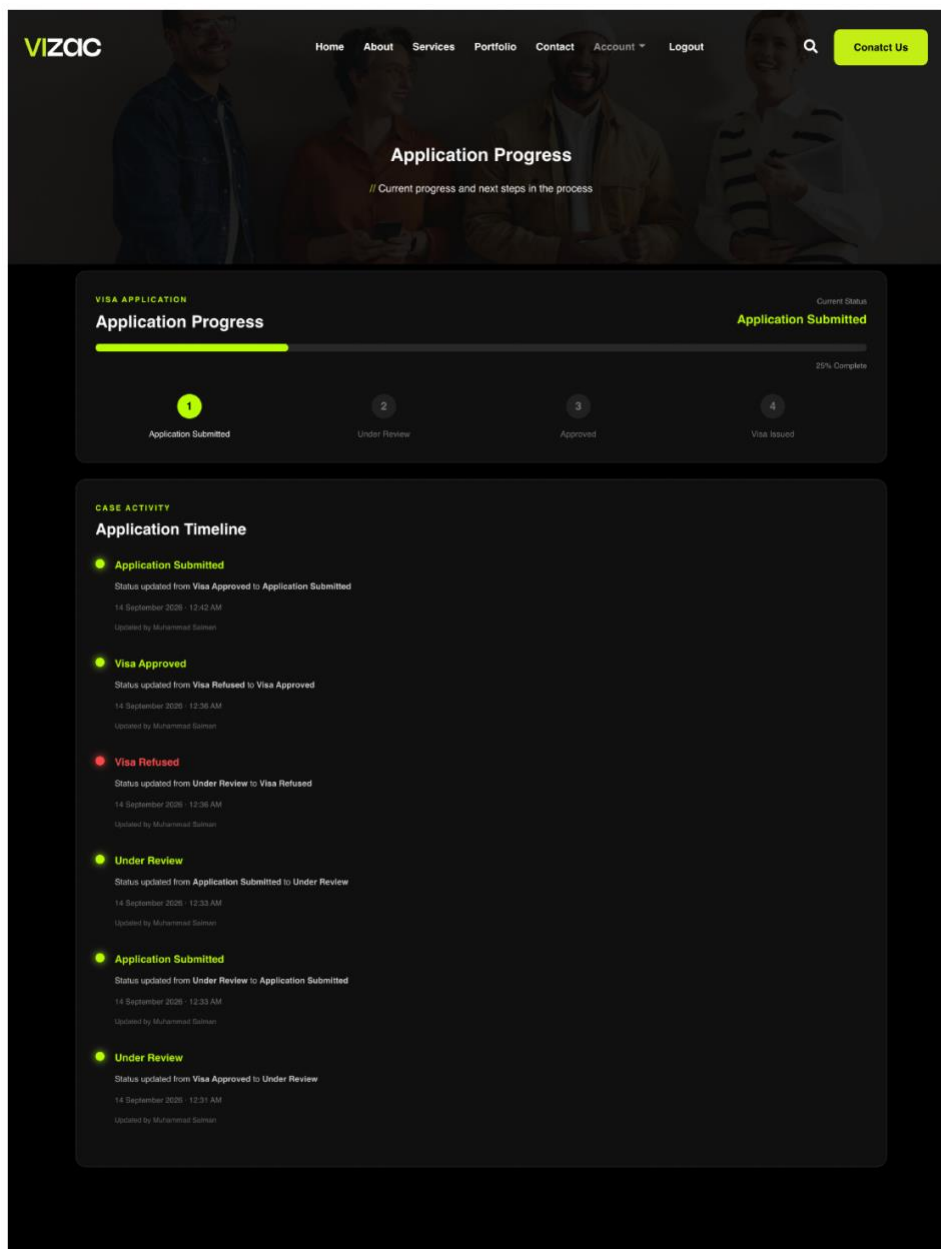


Figure 6. Application Progress page with current state and client-visible case activity timeline.

6.9 Data model and state representation

The implementation uses WordPress user metadata as a lightweight data model. The current application state is stored under `application_status`. The history is stored under `vizac_application_status_history` as an array of transition records. This is appropriate for a prototype in which each authenticated user has one principal tracked application state, but it would become limiting if the consultancy needed to manage multiple concurrent applications for the same client, complex document sets, assignments to multiple consultants or reporting across thousands of cases.

A more scalable future version could introduce a dedicated custom post type or custom database table for cases, with a case identifier, client relationship, status history and document references. The current model was intentionally kept simple to demonstrate the workflow without prematurely introducing a larger data architecture.

6.10 Security and maintainability controls

The plugin includes several practical controls. Unauthenticated users do not receive tracking information; output is derived from the current authenticated user; user-supplied values added to the history are sanitised; numeric IDs are normalised; and dynamic labels and colours are escaped before output. The history also improves accountability by retaining the updater identity. In addition, moving the code out of the theme reduces the chance that presentation-layer maintenance will silently remove application logic.

These controls are not sufficient for a production security claim. A production release should add explicit capability checks around administrative status changes, test least-privilege roles, consider stronger authentication for staff, review file-upload handling, enable secure transport and backups, establish monitoring and logging, and perform vulnerability assessment. WordPress and all third-party plugins would also need a defined patching process.

7. Testing and Evaluation

Testing concentrated on the project behaviours that were most important to the client workflow. Because the project did not include an external testing panel, the evidence is best described as developer-led functional and integration testing. The tests were repeated after restoration and after moving the status code from the theme into the custom plugin.

7.1 Functional and integration testing

Test	Scenario	Method	Expected / observed behaviour	Result
T01	Unauthenticated tracker access	Open tracking shortcode while logged out	Login guidance shown; no client status exposed	Pass
T02	Authenticated user context	Open tracker while logged in as a client	Current user's application status is loaded	Pass
T03	Submitted state	Set application_status to submitted	25% display; Submitted highlighted	Pass
T04	Review state	Set application_status to review	50% display; Under Review highlighted and guidance shown	Pass
T05	Approved state	Set application_status to approved	75% display; approval message shown	Pass
T06	Issued state	Set application_status to issued	100% display; completion message shown	Pass
T07	Refusal branch	Set application_status to refused	Red terminal refusal branch and refusal guidance shown	Pass

Test	Scenario	Method	Expected / observed behaviour	Result
T08	History capture	Change a user's status	Old/new status, timestamp and updater appended to history	Pass
T09	History display	Reload tracking page after several changes	Timeline lists recent transitions and updater name	Pass
T10	Shortcode registration	Load page containing [application_progress_bar]	Rendered progress interface replaces literal shortcode	Pass
T11	Booking integration	Open booking form after restoring site	Services became available after plugin update	Pass after maintenance
T12	PHP syntax	Run php -l against plugin file	No syntax errors	Pass

7.2 Usability review

The final interface was reviewed against the project's main user tasks rather than against a formal participant score. Public pages, login, registration, account, application submission and tracking share a consistent visual language. The progress page gives priority to current status, then stage position, then explanatory guidance and historical activity. This ordering supports rapid interpretation without requiring the client to understand WordPress or internal status codes.

Potential usability limitations remain. The desktop screenshots do not constitute a complete accessibility audit; keyboard navigation, screen-reader behaviour, colour contrast and error-message quality should be tested systematically. The application form is information-dense and may benefit from progressive disclosure or a multi-step format on smaller screens. A future study should use representative clients and a recognised questionnaire such as SUS together with task-completion observation.

7.3 Security and privacy review

Area	Current position	Evaluation
Authenticated access	Implemented	Tracker checks login state before returning client-specific output.
User record selection	Implemented	Uses the current authenticated WordPress user ID rather than a URL-supplied client ID.
Input sanitisation	Implemented in custom history logic	Status values are sanitised before being written to history; IDs are normalised.
Output escaping	Implemented in tracker/timeline	Dynamic labels and attributes are escaped before rendering.
Audit trail	Implemented	Previous/new status, time and updater are retained for status changes.
Role/capability model	Partial / requires further review	Production use should explicitly define which staff roles may alter case status.
MFA / privileged authentication	Not evaluated	Recommended for staff accounts in a production deployment.

Area	Current position	Evaluation
Penetration testing	Not performed	Independent web-application security testing is required before production claims.
Data retention and deletion	Not formalised	Production use requires retention, deletion and privacy procedures.
Secure document management	Not yet implemented	Future feature should use controlled storage and access rather than public media URLs.

7.4 Evaluation limitations

- No independent end-user study was conducted, so perceived usefulness and usability have not been statistically measured.
- No formal penetration test, static-analysis audit or threat-modelling exercise was completed.
- The development screenshots come from a local environment and do not demonstrate production hosting controls, TLS configuration or backup procedures.
- The current data model assumes one principal tracked application state per user and is not yet a full multi-case case-management system.
- Third-party plugin behaviour remains a dependency and must be retested after updates or migration.

7.5 Maintainability and portability evaluation

Maintainability improved substantially when the progress logic was moved from the Hello Elementor theme functions file into the dedicated VIZAC Client Portal plugin. Theme files primarily control presentation and can be replaced during redesign or updates; embedding business rules there couples the workflow to an unrelated presentation dependency. The plugin boundary makes the custom contribution easier to identify, back up, version-control and reuse.

The current plugin remains intentionally simple, with tracking, history and presentation in one PHP file. This is acceptable for a small prototype, but future development should separate responsibilities into modules such as progress-tracker.php, status-history.php, notifications.php and documents.php, with CSS and JavaScript moved into asset files. Such separation would improve testability and reduce the risk that a change to visual output affects data recording.

Portability is also constrained by external dependencies. The plugin expects the ACF application_status field to exist and assumes specific status values. A reusable public plugin should include configuration or activation checks, define supported dependencies, fail gracefully when required fields are absent, and provide migration documentation. These issues are not failures of the prototype; they are the engineering work required to move from a site-specific solution to a distributable software product.

8. Results and Discussion

Against the stated objectives, the project produced a complete demonstrable workflow rather than a collection of disconnected pages. Public consultancy information leads into registration and login; authentication establishes a user context; the user context supports profile and application functionality; the ACF status provides administratively controlled state; and the VIZAC plugin converts that state into client-facing progress and history. This end-to-end connection is important because the value of a client portal lies in the integration of these parts rather than in any single page.

The project achieved its central technical objective: the restored WordPress site operates as a consultancy portal in which an authenticated client can access a dedicated application-progress interface tied to their WordPress account. The completed tracker represents the case as a sequence of meaningful states rather than as free text, supports both positive and refusal outcomes, and presents a case-activity history beneath the current status. The final screenshots demonstrate a consistent visual integration between the public site and the tracking component.

The strongest result is not the percentage bar itself but the combination of state modelling and history. A simple progress widget could have been created entirely in page-builder content, but it would not respond reliably to administrative status changes or preserve previous transitions. The custom plugin establishes a clearer separation between administrative data, business logic and presentation. This is a more defensible software-engineering contribution and provides a pathway towards generalisation as a reusable consultancy plugin.

The project also demonstrates the benefits and constraints of using WordPress as an application platform. Development was accelerated by existing account, page, field and booking functionality. At the same time, plugin compatibility became a real operational issue, as shown by the booking form failure after restoration. This trade-off is consistent with the wider literature on information-system quality: a system is only successful if the integrated service remains available, understandable and maintainable, not merely because each component exists individually.

From a user-experience perspective, VIZAC addresses the core transparency problem identified at the beginning of the project. A client can view the current stage and recent history without contacting the consultant for every routine update. The interface also gives context-sensitive guidance for under-review, approved, issued and refused states. This is aligned with research that links transparency and self-service capability with service quality and user satisfaction. However, because no formal user study was conducted, the project cannot claim measured reductions in client contact or statistically demonstrated improvements in satisfaction.

From a security and governance perspective, the authenticated user context and audit-history design are useful foundations, but they also reveal future research questions. A production system would need to examine role-based access, secure document handling, data minimisation, retention, multi-case modelling, logging and independent security testing. These concerns make the project relevant beyond web design: it sits at the intersection of human-centred service delivery, information governance and web-application engineering.

9. Conclusion

This project designed, developed and evaluated VIZAC as a WordPress-based consultancy client portal intended to improve visibility of application progress. The platform combines public consultancy content with registration, authentication, profile management, application intake, booking and a user-specific tracking interface. The key custom contribution is the VIZAC Client Portal plugin, which retrieves a client's ACF-managed status, converts it into an understandable progress model, handles approval, issuance and refusal as distinct domain outcomes, and records status transitions with timestamps and updater identities.

The project shows that WordPress can support more specialised case-management behaviour when its user, metadata, hook and shortcode capabilities are combined with carefully scoped custom code. Moving the tracking logic from the theme into a standalone plugin improved maintainability and aligned the implementation more closely with WordPress development practice. The project also demonstrated the

importance of integration testing, as third-party plugin compatibility affected booking functionality after restoration.

The evaluation indicates that the prototype meets its core functional objectives, but the report deliberately avoids overstating the result. The system has not undergone independent usability testing, penetration testing or production data-protection assessment, and it currently models one principal application state per user. These limitations are significant, yet they are also productive: they define a clear roadmap for future work and identify research questions concerning secure self-service portals, auditability and human-centred case transparency.

10. Recommendations and Future Work

Future development should prioritise production readiness rather than adding cosmetic features. The highest-value technical improvements are: a dedicated case data model that can support multiple applications per client; document management with controlled access and retention rules; stronger role/capability enforcement for consultants; automated but carefully worded notifications when status changes; independent security testing; and formal usability evaluation with representative users.

For portability, the custom plugin should also be refactored into separate modules for status tracking, history, notifications and documents, with configurable status labels rather than consultancy-specific hard-coding. A future research-oriented version could compare different ways of presenting case progress and measure their effects on perceived transparency, trust, support demand and usability. Such a study would turn the current design artefact into a stronger empirical research platform.

References

- Brooke, J. (1996) 'SUS: A quick and dirty usability scale', in Jordan, P.W., Thomas, B., McClelland, I.L. and Weerdmeester, B. (eds.) *Usability Evaluation in Industry*. London: Taylor & Francis, pp. 189-194.
- Chan, F.K.Y., Brown, S.A. and Venkatesh, V. (2021) 'Service Design and Citizen Satisfaction with E-Government Services: A Multidimensional Perspective', *Public Administration Review*, 81(5). doi:10.1111/puar.13308.
- Davis, F.D. (1989) 'Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology', *MIS Quarterly*, 13(3), pp. 319-340. doi:10.2307/249008.
- DeLone, W.H. and McLean, E.R. (2003) 'The DeLone and McLean Model of Information Systems Success: A Ten-Year Update', *Journal of Management Information Systems*, 19(4), pp. 9-30. doi:10.1080/07421222.2003.11045748.
- Amazon Web Services (2026) What is SDLC? - Software Development Lifecycle Explained. Available at: <https://aws.amazon.com/what-is/sdlc/> (Accessed: 19 September 2026).
- Beck, K. et al. (2001) Manifesto for Agile Software Development. Available at: <https://agilemanifesto.org/> (Accessed: 19 September 2026).
- Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design Science in Information Systems Research', *MIS Quarterly*, 28(1), pp. 75-105.
- Information Commissioner's Office (2026) A guide to the data protection principles. Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/data-protection-principles/> (Accessed: 14 September 2026).

Information Commissioner's Office (2026) Security outcomes: A guide to data security. Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/security/> (Accessed: 14 September 2026).

OWASP Foundation (2021) OWASP Top 10:2021. Available at: <https://owasp.org/Top10/> (Accessed: 14 September 2026).

Papadomichelaki, X. and Mentzas, G. (2012) 'e-GovQual: A multiple-item scale for assessing e-government service quality', *Government Information Quarterly*, 29(1), pp. 98-109. doi:10.1016/j.giq.2011.08.011.

Parasuraman, A., Zeithaml, V.A. and Malhotra, A. (2005) 'E-S-QUAL: A Multiple-Item Scale for Assessing Electronic Service Quality', *Journal of Service Research*, 7(3), pp. 213-233. doi:10.1177/1094670504271156.

WordPress.org (2023) Plugin Developer Handbook. Available at: <https://developer.wordpress.org/plugins/> (Accessed: 14 September 2026).

WordPress.org (2024) Hooks - Plugin Handbook. Available at: <https://developer.wordpress.org/plugins/hooks/> (Accessed: 14 September 2026).

WordPress.org (2026) Hardening WordPress. Available at: <https://developer.wordpress.org/advanced-administration/security/hardening/> (Accessed: 14 September 2026).

WP Engine / Advanced Custom Fields (2024) `get_field_object()`. Available at: https://www.advancedcustomfields.com/resources/get_field_object/ (Accessed: 14 September 2026).

BookingPress (2026) BookingPress Setup Wizard. Available at: <https://www.bookingpressplugin.com/documents/bookingpress-setup-wizard/> (Accessed: 14 September 2026).

Appendix A. Public Repository and Reproducibility Notes

The public repository is intended to provide a reproducible view of the custom VIZAC contribution without publishing a complete WordPress installation, database backup or third-party proprietary packages. The repository is available at <https://github.com/itxmsalman/vizac-client-portal> and documents the VIZAC Client Portal v1.0.0 release, source code and sanitised project screenshots. Screenshots containing personal contact details or profile data are excluded from the public version. A suitable public structure is: The repository README should explain dependencies such as WordPress and ACF and state clearly that the project is a prototype/portfolio artefact rather than an official government tracking integration.

```
vizac-client-portal/  
├── README.md  
├── .gitignore  
├── vizac-client-portal/  
│   └── vizac-client-portal.php  
├── docs/  
│   ├── PROJECT_SUMMARY.md  
│   └── TESTING.md  
└── screenshots/  
    └── (sanitised screenshots only)
```

- Do not upload `wp-config.php`, database exports, `.wpress` backups, `.env` files, passwords, API keys or server configuration containing secrets.
- Do not upload paid/proprietary third-party plugin packages unless their licence explicitly permits redistribution.